Guia explicativa paso a paso

Construccion del Dashboard Ejecutivo de Bajas TI

Universidad Autonoma de Occidente - Unidad de Servicio al Usuario

Este documento describe, paso a paso y con diagramas, como se construyo la aplicacion web (Dashboard Ejecutivo) a partir de las solicitudes de baja de activos (DITI), explica el script de Python empleado para transformar los datos y detalla las herramientas de terceros utilizadas.

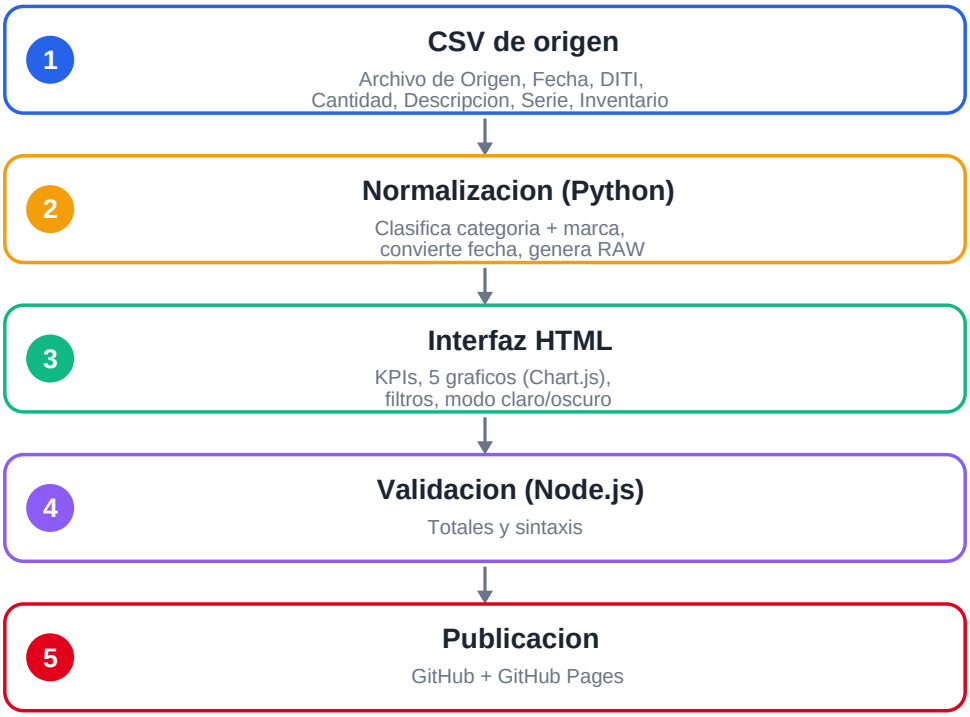
Contenido

- 1. Vision general del flujo (diagrama)
- 2. Paso 1 - Lectura de los datos de origen
- 3. Paso 2 - Normalizacion con Python (script + pipeline)
- 4. Paso 3 - Diseno de la app HTML (maqueta)
- 5. Paso 4 - Validacion de los datos
- 6. Paso 5 - Publicacion en GitHub y Pages
- 7. Herramientas de terceros utilizadas
- 8. Resumen de resultados

1. Vision general del flujo

El objetivo fue convertir un listado tabular de bajas de equipos en un tablero ejecutivo interactivo, autocontenido en un unico archivo HTML. El proceso siguio cinco etapas, representadas en el siguiente diagrama de flujo:

Diagrama 1. Flujo general del proyecto (de los datos a la publicacion).



2. Paso 1 - Lectura de los datos de origen

La fuente final fue un archivo CSV exportado desde Excel. Cada fila representa un activo individual dado de baja. Ejemplo de las primeras lineas:

```
Archivo de Origen,Fecha,Consecutivo,Cantidad,Descripcion,Serie No.,No. inventario
Bajas 2025.pdf,16/1/2025,DITI-8.6-01,1,TELEFONO IP GAMA MEDIA 1608,10WZ25052647,98975
Bajas 2025.pdf,21/4/2025,DITI-4.6-23,1,"Imac 21""",QP05103PDB6,100033
```

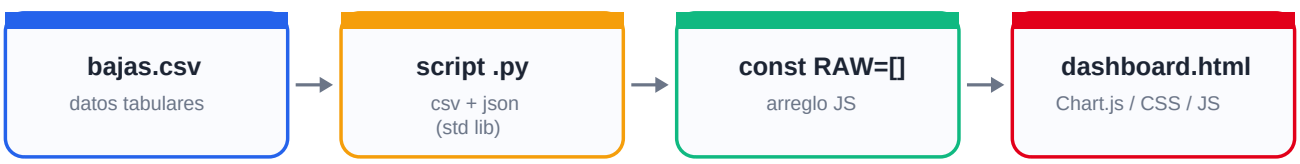
Caracteristicas que condicionaron el procesamiento:

- Descripciones con comas van entrecomilladas (CSV estandar).
- Cantidad puede ser mayor que 1 (lotes de mouse, teclados, etc.).
- Algunos campos de serie o inventario vienen vacios.
- Fechas en formato d/m/aaaa (se convierten a aaaa-mm-dd).
- Dos archivos de origen: Bajas 2025.pdf y bajas 2026.pdf.

3. Paso 2 - Normalizacion con Python

Un script en Python (solo libreria estandar: csv y json) lee el CSV, convierte la fecha, clasifica cada fila por categoria y marca mediante palabras clave, y exporta un arreglo JavaScript (const RAW = [...]) que el dashboard consume. El flujo de datos es el siguiente:

Diagrama 2. Pipeline de datos: del CSV al arreglo RAW dentro del HTML.



3.1 Lectura del CSV

```
import csv, json
rows = []
```

```
with open('bajas.csv', encoding='utf-8') as f:
    r = csv.DictReader(f)          # lee por nombre de columna
    for row in r:
        rows.append(row)           # cada 'row' es un diccionario
```

3.2 Conversion de la fecha

```
def parse_date(d):
    dd, mm, yy = d.strip().split('/')
    return f'{int(yy):04d}-{int(mm):02d}-{int(dd):02d}'
# '16/1/2025'  ->  '2025-01-16'
```

3.3 Clasificacion por categoria (palabras clave)

```
def categorize(desc):
    d = desc.lower()
    if any(k in d for k in ['celular','galaxy','alcatel','nokia']):
        return 'Celular'
    if 'portatil' in d or 'macbook' in d or 'latitude' in d:
        return 'Portatil'
    if any(k in d for k in ['cpu','optiplex','imac','aio','desktop']):
        return 'CPU/Computador'
    if 'monitor' in d or 'pantalla' in d:   return 'Monitor'
    if 'teclado' in d:                       return 'Teclado'
    if 'mouse' in d:                         return 'Mouse'
    if 'diadema' in d or 'parlante' in d:    return 'Diadema/Audio'
    if 'telefono' in d or 'avaya' in d:      return 'Telefono'
    if 'ipad' in d or 'tablet' in d:         return 'Tablet'
    if any(k in d for k in ['cisco','switch','servidor']):
        return 'Red/Networking'
    return 'Otros'
```

3.4 Deteccion de la marca

```
def brand(desc):
    d = desc.lower()
    pairs = [('dell','Dell'),('imac','Apple'),('macbook','Apple'),
             ('hp ','HP'),('avaya','Avaya'),('samsung','Samsung'),
             ('cisco','Cisco'),('logitech','Logitech'),('xkim','Xkim')]
    for clave, marca in pairs:
        if clave in d: return marca
    return 'Generico'
```

3.5 Exportacion del arreglo RAW

```
out = []
for row in rows:
    desc = row['Descripcion del elemento'].strip()
    if not desc: continue
    out.append({
      'src': row['Archivo de Origen'].strip(),
      'doc': row['Consecutivo'].strip(),
      'date': parse_date(row['Fecha']),
      'qty': int(row['Cantidad'] or '1'),
      'cat': categorize(desc), 'brand': brand(desc),
      'desc': desc, 'serie': row['Serie No.'].strip(),
      'inv': row['No. inventario'].strip() })
with open('_raw.js','w',encoding='utf-8') as f:
    f.write('const RAW = [\n')
    for o in out: f.write('  '+json.dumps(o,ensure_ascii=False)+',\n')
    f.write('];\n')
```

4. Paso 3 - Diseno de la aplicacion HTML

El dashboard es un unico archivo .html con CSS y JavaScript embebidos; solo depende de Chart.js (via CDN). La siguiente maqueta muestra la disposicion de sus bloques:

Diagrama 3. Maqueta (wireframe) del layout del dashboard.

Componentes de la interfaz

- Encabezado con marca institucional y boton de modo claro/oscuro.
- Fila de filtros: archivo de origen, mes, marca, documento DITI, buscador y boton Limpiar; mas chips de categoria.
- Seis tarjetas de KPI con metricas agregadas en tiempo real.
- Cinco graficos (Chart.js): dona por categoria, barras por marca, tendencia mensual (barras+linea), top descripciones y unidades por documento.
- Tabla de detalle filtrable con todos los activos.

Cada cambio en un filtro ejecuta getFiltered() + render(), que recalculan KPIs, graficos y tabla sin recargar la pagina.

5. Paso 4 - Validacion de los datos

Antes de publicar se verificaron totales y sintaxis con Node.js, extrayendo el arreglo RAW del HTML:

```
const fs = require('fs');
const html = fs.readFileSync('dashboard-bajas-uao.html','utf8');
const m = html.match(/const RAW = \[([\s\S]*?)\n\];/);
const RAW = eval('[' + m[1] + ']');
```

```
console.log('Registros:', RAW.length);
console.log('Unidades :', RAW.reduce((s,r)=>s+r.qty,0));
console.log('Documentos:', new Set(RAW.map(r=>r.doc)).size);
```

6. Paso 5 - Publicacion en GitHub y Pages

```
git init
git add dashboard-bajas-uao.html README.md bajas.csv
git commit -m 'Dashboard Ejecutivo de Bajas TI'
git push -u origin main
```

Para exponerlo como sitio web se activa, en Settings > Pages, la rama main / carpeta raiz. Queda accesible en una URL del tipo:

```
https://<usuario>.github.io/<repositorio>/dashboard-bajas-uao.html
```

7. Herramientas de terceros utilizadas

- Chart.js (v4, via CDN jsdelivr): unica libreria de terceros de la aplicacion; renderiza los cinco graficos interactivos.
- Python 3 (libreria estandar: csv y json): normalizacion. NO se usaron librerias de terceros de Python (ni pandas ni openpyxl).
- Node.js (incluido en el entorno): solo validacion de totales y sintaxis.
- Git y GitHub + Pages: control de versiones, alojamiento y publicacion.
- Para ESTE PDF y sus diagramas: generador propio en Python puro (libreria estandar), que escribe la estructura PDF y dibuja los diagramas como graficos vectoriales. Sin pandoc, wkhtmltopdf, reportlab ni fpdf.

Tecnologias base del dashboard (sin instalacion): HTML5, CSS3 y JavaScript (ES6).

Nota: los diagramas de este documento son esquematicos (dibujados de forma vectorial) ya que el entorno no dispone de navegador para capturas de pantalla reales; representan fielmente la estructura y disposicion del dashboard.

8. Resumen de resultados

- 935 lineas de activo procesadas.
- 1.338 unidades totales dadas de baja.
- 106 solicitudes (documentos DITI).
- 11 categorias y 41 marcas detectadas automaticamente.
- 2 archivos de origen: Bajas 2025.pdf y bajas 2026.pdf.
- Periodo cubierto: enero 2025 a octubre 2026.